



哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

高级语言程序设计

High-level Language Programming

Lecture 2 Basic syntax of C++

Weizheng Zhang (zhangweizheng@hit.edu.cn)
School of Information Science and Technology

Basics syntax of C++

Course Overview

- Variable
 - Identifiers
 - Data types
 - Literals
- Variable properties
 - Name, type, and value
 - Memory storage
- Variable definition and assignment

Let's start with a "hello world" example



The screenshot shows an online C++ compiler interface. At the top, there's a blue header with the text "</> ONLINE CPP". Below this, there's a toolbar with icons for file operations (folder, save, share) and a dropdown menu set to "C++". The main area is a code editor with a file named "main.cpp" open. The code is as follows:

```
1 #include<iostream>
2
3 int main()
4 {
5     int my_number = 2024;
6
7     char my_text[20] = "Hello world ";
8
9     std::cout << my_text << my_number;
10
11     return 0;
12 }
13
14
```

Below the code editor, there's a status bar showing "Ln: 14, Col: 1". To the right of the code editor, there are buttons for "Run" (with a green play icon), "Share", and a text input field for "Command Line Arguments". Below these, there's a terminal window showing the output of the program:

```
Hello world 2024

** Process exited - Return Code: 0 **
```

Try it yourself on <https://www.onlinegdb.com>

Let's start with a "hello world" example

</> ONLINE CPP

C++

main.cpp +

```
1 #include<iostream>
2
3 int main()
4 {
5     int my_number = 2024;
6     char my_text[20] = "Hello world ";
7
8     std::cout << my_text << my_number;
9
10
11     return 0;
12 }
13
14
```

Ln: 14, Col: 1

Run

Share

Command Line Arguments

Hello world 2024

** Process exited - Return Code: 0 **

We create two variables (what we learn today)

Try it yourself on <https://www.onlinegdb.com>

C++ hello world example

```
</> ONLINE CPP

main.cpp +
1  #include<iostream>
2
3  int main()
4  {
5      int my_number = 2024;
6
7      char my_text[20] = "Hello world ";
8
9      std::cout << my_text << my_number;
10
11     return 0;
12 }
13
14

Ln: 14, Col: 1

Run Share Command Line Arguments

Hello world 2024

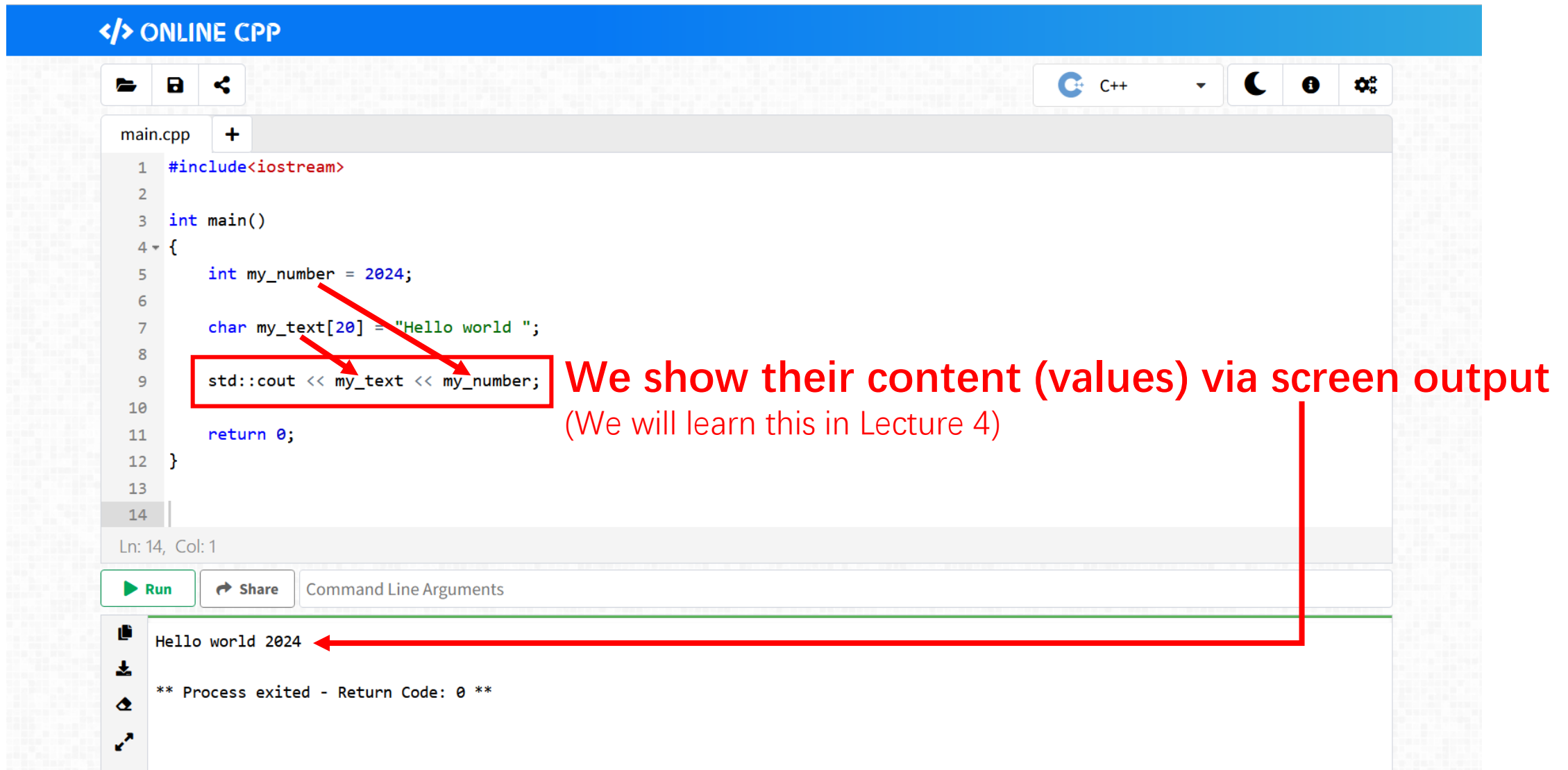
** Process exited - Return Code: 0 **
```

Like a
"Container"

2024
my_number

Hello world
my_text

Let's start with a "hello world" example



The screenshot displays an online C++ compiler interface. At the top, a blue header bar contains the text "</> ONLINE CPP". Below this, a toolbar includes icons for file operations and a language selector set to "C++". The main editor area shows a file named "main.cpp" with the following code:

```
1 #include<iostream>
2
3 int main()
4 {
5     int my_number = 2024;
6
7     char my_text[20] = "Hello world ";
8
9     std::cout << my_text << my_number;
10
11     return 0;
12 }
```

A red box highlights the line `std::cout << my_text << my_number;`. Red arrows point from this line to the output section below. The output section shows the result of running the program: "Hello world 2024". Below the output, it states "** Process exited - Return Code: 0 **".

We show their content (values) via screen output
(We will learn this in Lecture 4)

Try it yourself on <https://www.onlinegdb.com>

C++ hello world example

```
</> ONLINE CPP

main.cpp +
1  #include<iostream>
2
3  int main()
4  {
5      int my_number = 2024;
6
7      char my_text[20] = "Hello world ";
8
9      std::cout << my_text << my_number;
10
11     return 0;
12 }
13
14

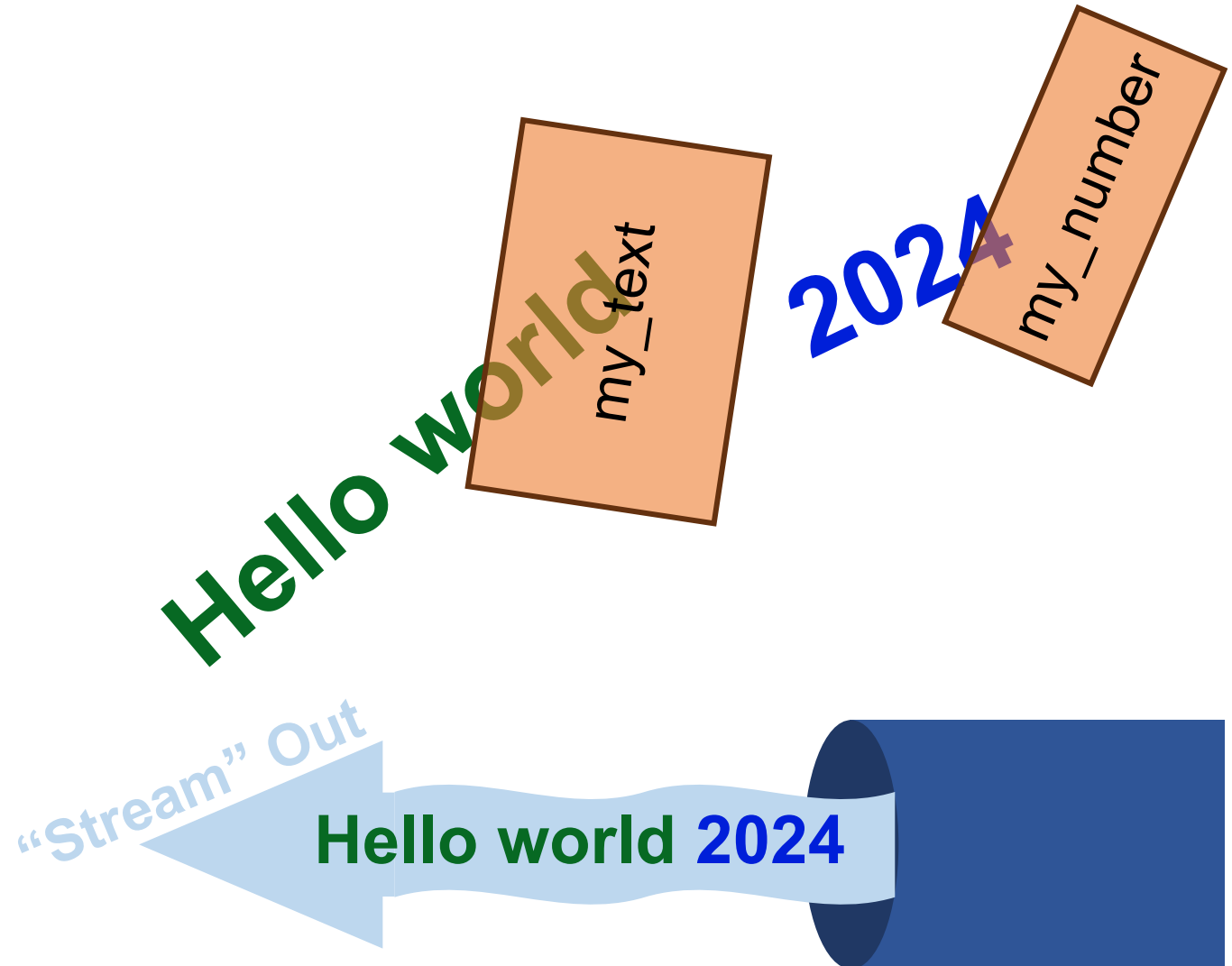
Ln: 14, Col: 1

Run Share Command Line Arguments

Hello world 2024

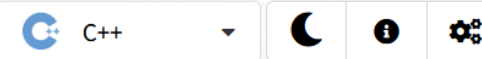
** Process exited - Return Code: 0 **
```

Try it yourself on <https://www.onlinegdb.com>



C++ hello world example

</> ONLINE CPP



main.cpp +

```
1 #include<iostream>
2
3 int main()
4 {
5     int my_number = 2024;
6
7     char my_text[20] = "Hello world ";
8
9     std::cout << my_text << my_number;
10
11     return 0;
12 }
13
14
```

Identifiers (names)

Literals (values)

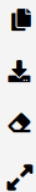
Data types

Ln: 14, Col: 1

Run

Share

Command Line Arguments

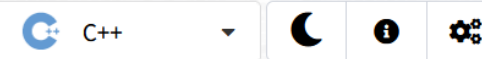


Hello world 2024

** Process exited - Return Code: 0 **

C++ hello world example

</> ONLINE CPP

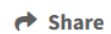


main.cpp +

```
1 #include<iostream>
2
3 int main()
4 {
5     int my_number = 2024;
6
7     char my_text[20] = "Hello world ";
8
9     std::cout << my_text << my_number;
10
11     return 0;
12 }
13
14
```

Identifiers used to name entities, such as constants, variables, or functions

Ln: 14, Col: 1



Command Line Arguments



Hello world 2024



** Process exited - Return Code: 0 **

Identifiers

- Valid identifier is sequence of one or more **letters, digits, and underscore characters** that does **not begin with a digit**
- Identifiers that **begin with underscore** or **contain double underscores** are reserved for use by C++ implementation and should be avoided whenever possible
- Example of valid identifiers:
 - `event_counter`
 - `eventCounter`
 - `sqrt_2`
 - `f_o_o_b_a_r_4_2`

Identifiers

- Identifiers are **case sensitive**

For example, the following are distinct identifiers:

counter

cOuNtEr

- Identifiers cannot be any of reserved keywords

alignas	constexpr	mutable	switch
alignof	constinit	namespace	template
and	const_cast	new	this
and_eq	continue	noexcept	thread_local
asm	decltype	not	throw
auto	default	not_eq	true
bitand	delete	nullptr	try
bitor	do	operator	typedef
bool	double	or	typeid
break	dynamic_cast	or_eq	typename
case	else	private	union
catch	enum	protected	unsigned
char	explicit	public	using
char8_t	export	register	virtual
char16_t	extern	reinterpret_cast	void
char32_t	false	requires	volatile
class	float	return	wchar_t
co_await	for	short	while
co_return	friend	signed	xor
co_yield	goto	sizeof	xor_eq
compl	if	static	final*
concept	inline	static_assert	import*
const	int	static_cast	module*
constexpr	long	struct	override*

* Note: context sensitive

Identifiers

- Identifiers are better to be with good naming convention.
 - Function naming, variable naming, and file naming should be descriptive, with fewer abbreviations. Use descriptive naming wherever possible to make code easy to understand.

```
int student_num; // number of students
int num_errors;  // number of errors

int n;           // not have any meaning
int nerr;        // ambiguous abbreviations
```

- Camel case: The first letter of the variable name is lowercase, and the first letter of each subsequent word is capitalized without underscores.

```
int studentNum;
int numErrors;
```

Identifiers

- Identifiers are better to be with good naming convention.
 - Underscore case: Words are separated by underscores. Variable names are usually in all lowercase.

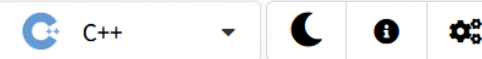
```
int student_num;  
int num_errors;
```

- Use easily understandable abbreviations: If abbreviations are necessary to simplify variable names, ensure that the abbreviations are common, easily understandable, and comment in the code to explain their meanings.

```
string fstName; // first name  
string lstName; // last name
```

Data types

</> ONLINE CPP



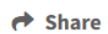
main.cpp +

```
1 #include<iostream>
2
3 int main()
4 {
5     int my_number = 2024;
6
7     char my_text[20] = "Hello world ";
8
9     std::cout << my_text << my_number;
10
11     return 0;
12 }
13
14
```

Data types

determine the **allocated memory space**,
the **storage format of data**,
the **valid range of values**, and
the **types of operations it can participate in**

Ln: 14, Col: 1



Command Line Arguments



Hello world 2024



** Process exited - Return Code: 0 **

Basic data types

- Boolean type **bool**
- Integer type (signed and unsigned)
 - signed: $\pm$
 - unsigned: only +

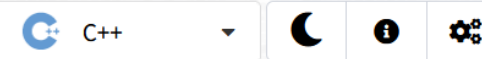
signed char	unsigned char
signed short int	unsigned short int
signed int	unsigned int
signed long int	unsigned long int
signed long long int	unsigned long long int
- Floating-point types
 - float 32 bits **float**
 - double 64 bits **double**
 - long double **long double**
- Character types **char**
- Void type (incomplete/valueless) **void**
- Pointer type **int*** **char***
- Array type
 - `int ages[10]` `char my_text[20]`

We will learn more details about the basic data types later in this lecture...

We will learn in future lectures...

Literals

</> ONLINE CPP



main.cpp +

```
1 #include<iostream>
2
3 int main()
4 {
5     int my_number = 2024;
6
7     char my_text[20] = "Hello world ";
8
9     std::cout << my_text << my_number;
10
11     return 0;
12 }
13
14
```

Literals

Ln: 14, Col: 1



Command Line Arguments



Hello world 2024



** Process exited - Return Code: 0 **

Literals

- Literal (literal constant) is value written exactly as it is meant to be interpreted; it is a fixed value that the program may not alter
- Examples of literals:

"Hello, world"
"Bjarne"
'a'
'A'
123
123U
1'000'000'000
3.1415
1.0L

```
main.cpp +
1  #include<iostream>
2
3  int main()
4  {
5      int my_number = 2024;
6
7      char my_text[20] = "Hello world ";
8
9      std::cout << my_text << my_number;
10
11     return 0;
12 }
```

Literals

- Boolean literals: **true**
false
- Pointer literal: **nullptr**

Integer literal types

- **Prefixes** which indicates the **base**. For example, **0x**10 indicates the value 16 in **hexadecimal** having **prefix 0x**.
 - **Decimal-literal** (**base 10**):- **a non-zero decimal digit** followed by zero or more decimal digits(0, 1, 2, 3, 4, 5, 6, 7, 8, 9). For example, 56, 78.
 - **Hex-literal** (base 16):- **0x or 0X** followed by one or more hexadecimal digits(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, A, b, B, c, C, d, D, e, E, f, F). For example, 0x23A, 0Xb4C, 0xFEB.

Integer literal types

- **Prefixes** which indicates the **base**. For example, **0x10** indicates the value 16 in hexadecimal having prefix **0x**.
 - **Decimal-literal** (base 10):- a non-zero decimal digit followed by zero or more decimal digits(0, 1, 2, 3, 4, 5, 6, 7, 8, 9). For example, 56, 78.
 - **Hex-literal** (base 16):- **0x** or **0X** followed by one or more hexadecimal digits(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, A, b, B, c, C, d, D, e, E, f, F). For example, 0x23A, 0xb4C, 0xFEB.

Conversion from Hexadecimal to decimal:

$$0x10 = 1 * 16 + 0 = 16 \text{ (decimal)}$$

$$0x23A = 2 * 16^2 + 3 * 16 + A = 512 + 48 + 10 = 570 \text{ (decimal)}$$

$$0xFEAB = F * 16^3 + E * 16^2 + B = 3840 + 224 + 11 = 4075 \text{ (decimal)}$$

A	10
B	11
C	12
D	13
E	14
F	15

Integer literal types

- **Prefixes** which indicates the **base**. For example, **0x**10 indicates the value 16 in hexadecimal having prefix **0x**.
 - **Decimal-literal** (base 10):- a **non-zero decimal digit** followed by zero or more decimal digits(0, 1, 2, 3, 4, 5, 6, 7, 8, 9). For example, 56, 78.
 - **Hex-literal** (base 16):- **0x or 0X** followed by one or more hexadecimal digits(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, A, b, B, c, C, d, D, e, E, f, F). For example, 0x23A, 0xb4C, 0xFEa.
 - **Binary-literal** (base 2):- **0b or 0B** followed by one or more binary digits(0, 1). For example, 0b101, 0B111.
 - **Octal-literal** (base 8):- a **0** followed by zero or more octal digits(0, 1, 2, 3, 4, 5, 6, 7). For example, 045, 076, 06210.

Question: what are the values stored inside the following variables?

```
int d = 42;  
int o = 052;  
int x = 0x2a;  
int X = 0X2A;  
int b = 0b101010;
```

Integer literal types

- **Prefixes** which indicates the **base**. For example, **0x10** indicates the value 16 in hexadecimal having prefix **0x**.
 - **Decimal-literal** (base 10):- **a non-zero decimal digit** followed by zero or more decimal digits(0, 1, 2, 3, 4, 5, 6, 7, 8, 9). For example, 56, 78.
 - **Hex-literal** (base 16):- **0x or 0X** followed by one or more hexadecimal digits(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, A, b, B, c, C, d, D, e, E, f, F). For example, 0x23A, 0xb4C, 0xFEa.
 - **Binary-literal** (base 2):- **0b or 0B** followed by one or more binary digits(0, 1). For example, 0b101, 0B111.
 - **Octal-literal** (base 8):- **a zero** followed by zero or more octal digits(0, 1, 2, 3, 4, 5, 6, 7). For example, 045, 076, 06210.

Question: what are the values stored inside the following variables?

```
int d = 42;
int o = 052;      = 5 * 8 + 2 = 42
int x = 0x2a;     = 2 * 16 + A = 42
int X = 0X2A;
int b = 0b101010; = 1 * 25 + 1 * 23 + 1 * 21 = 32 + 8 + 2 = 42
```

Integer literal types

- **Suffixes** which indicates the **data type**. For example, 12345678901234**LL** indicates the value 12345678901234 as an long long integer having suffix **LL**.
 - **int**:- No suffix are required because integer constant are by default assigned as int data type.
 - **unsigned int**: character **u** or **U** at the end of integer constant.
 - **long int**: character **l** or **L** at the end of integer constant.
 - **unsigned long int**: character **ul** or **UL** at the end of integer constant.
 - **long long int**: character **ll** or **LL** at the end of integer constant.
 - **unsigned long long int**: character **ull** or **ULL** at the end of integer constant.

Floating-point literal types

Suffix	Type
None	double
f or F	float
l or L	long double

1.414
1.25e-8

1.25×10^{-8}

1.414f
1.25e-8f

1.25×10^{-8}

1.5L
1.25e-20L

1.25×10^{-20}

Suffix

Expressions with **e**xponents

s × r^j

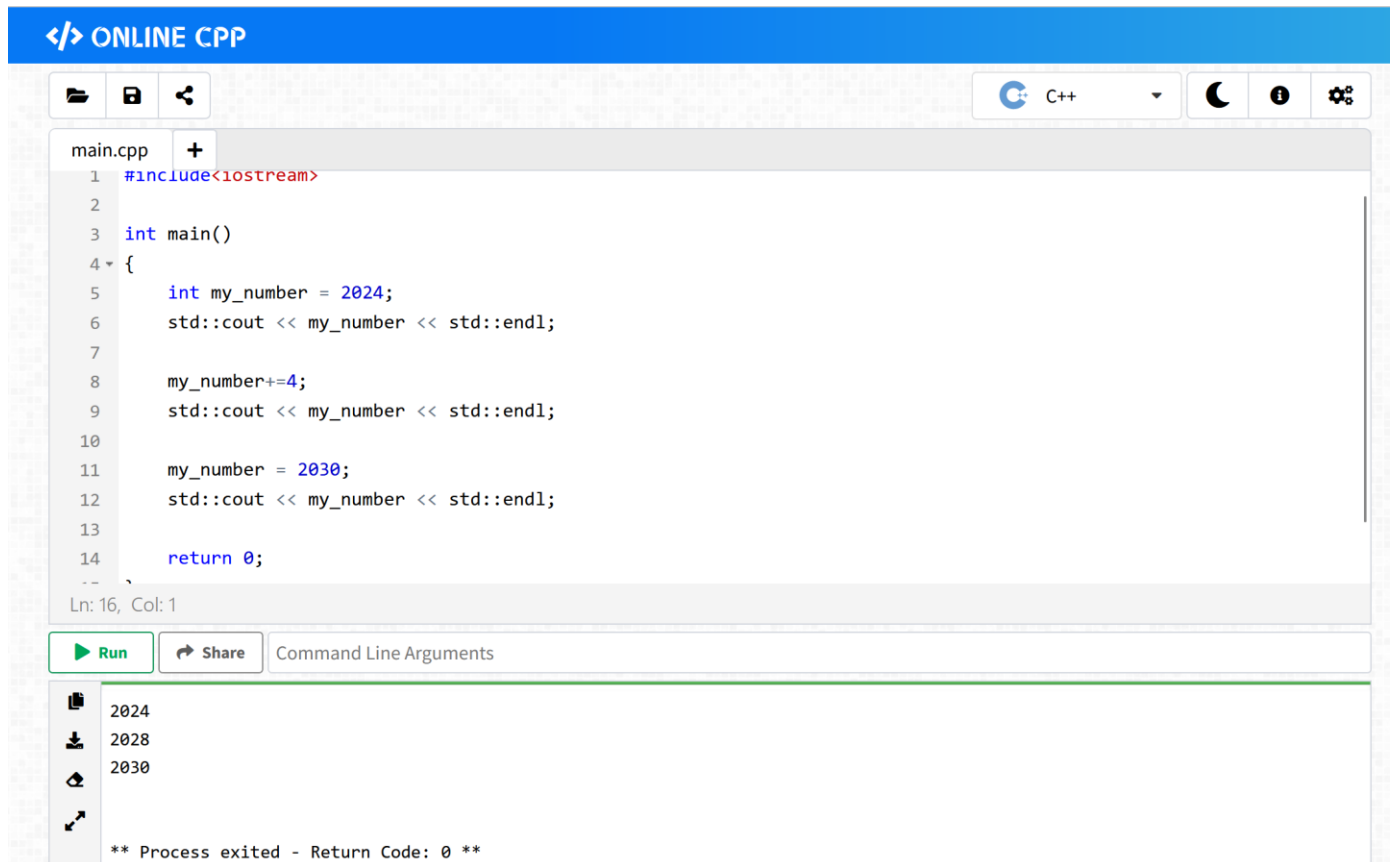
Significand s

Exponent j (how many times to use the number in a multiplication)

Radix r (base)

Constants and variables

- Constant: a fixed value that the program may not alter
- Variable: the value stored inside **can vary** during program execution



The screenshot shows an online C++ compiler interface. The code in `main.cpp` is as follows:

```
1 #include<iostream>
2
3 int main()
4 {
5     int my_number = 2024;
6     std::cout << my_number << std::endl;
7
8     my_number+=4;
9     std::cout << my_number << std::endl;
10
11     my_number = 2030;
12     std::cout << my_number << std::endl;
13
14     return 0;
15 }
```

The output of the program is displayed below the code editor:

```
2024
2028
2030
** Process exited - Return Code: 0 **
```

Variables

</> ONLINE CPP

<https://www.online-cpp.com/>



C++

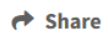


main.cpp



```
1 #include<iostream>
2
3 int main()
4 {
5     int my_number = 2024;
6
7     char my_text[20] = "Hello world ";
8
9     std::cout << my_text << my_number;
10
11     return 0;
12 }
13
14
```

Ln: 14, Col: 1



Command Line Arguments



Hello world 2024



** Process exited - Return Code: 0 **

Variable
data type

Variable name

Variable value

Variable address: where to store it in memory

Allocated memory space: determined by its
data type

Data types determine allocated memory space

```
</> ONLINE CPP

main.cpp +
1  #include<iostream>
2
3  int main()
4  {
5      int my_number = 2024;
6
7      char my_text[20] = "Hello world ";
8
9      std::cout << my_text << my_number;
10
11     return 0;
12 }
13
14

Ln: 14, Col: 1

Run Share Command Line Arguments

Hello world 2024

** Process exited - Return Code: 0 **
```

Variable address: 0x0025c010

Variable value: 0x000007e8

Physical memory

xxxxxxx
xxxxxxx
xxxxxxx
xxxxxxx
xxxxxxx
xxxxxxx
xxxxxxx
00000000
00000000
00000111
11101000
xxxxxxx
xxxxxxx
xxxxxxx
xxxxxxx
xxxxxxx
xxxxxxx
xxxxxxx
xxxxxxx
xxxxxxx
xxxxxxx
xxxxxxx
xxxxxxx

Allocated Memory for "int"

Basic data types

- Boolean type **bool**
- Integer type (signed and unsigned)

signed char	unsigned char
signed short int	unsigned short int
signed int	unsigned int
signed long int	unsigned long int
signed long long int	unsigned long long int

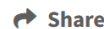
“signed” may be omitted for names of signed numerical integer types
- Floating-point types **float**
 double
 long double
- Character types **char**
(Note that **char** is distinct type from **signed char** and **unsigned char**)

Get the size of allocated memory space

- **sizeof** operator determines the size, in bytes, of a variable or data type;
- Can be used to get the size of classes, structures, unions and any other user defined data type.

```
main.cpp +
1  #include <iostream>
2
3  int main() {
4      std::cout << "Size of bool : " << sizeof(bool) << std::endl;
5      std::cout << "Size of char : " << sizeof(char) << std::endl;
6      std::cout << "Size of short int : " << sizeof(short int) << std::endl;
7      std::cout << "Size of int : " << sizeof(int) << std::endl;
8      std::cout << "Size of long int : " << sizeof(long int) << std::endl;
9      std::cout << "Size of long long int : " << sizeof(long long int) << std::endl;
10     std::cout << "Size of float : " << sizeof(float) << std::endl;
11     std::cout << "Size of double : " << sizeof(double) << std::endl;
12     std::cout << "Size of long double : " << sizeof(long double) << std::endl;
13
14     return 0;
15 }
```

Ln: 1, Col: 20

  Command Line Arguments

```
Size of bool : 1
Size of char : 1
Size of short int : 2
Size of int : 4
Size of long int : 8
Size of long long int : 8
Size of float : 4
Size of double : 8
Size of long double : 16
```

Basic data types

- Boolean type **bool** 1 Bytes
- Integer type (signed and unsigned)

signed char	unsigned char	1 Bytes
signed short int	unsigned short int	2 Bytes
signed int	unsigned int	4 Bytes
signed long int	unsigned long int	8 Bytes
signed long long int	unsigned long long int	8 Bytes
- Floating-point types

float	4 Bytes
double	8 Bytes
long double	16 Bytes
- Character types **char** 1 Bytes
(Note that **char** is distinct type from **signed char** and **unsigned char**)



Character encoding

ASCII Chart

Character encoded in 1 Byte

ASCII stands for American Standard Code for Information Interchange, and it defines a particular way to represent English characters (plus a few other symbols) as numbers between 0 and 127

Code	Symbol	Code	Symbol	Code	Symbol	Code	Symbol
0	NUL (null)	32	(space)	64	@	96	`
1	SOH (start of header)	33	!	65	A	97	a
2	STX (start of text)	34	"	66	B	98	b
3	ETX (end of text)	35	#	67	C	99	c
4	EOT (end of transmission)	36	\$	68	D	100	d
5	ENQ (enquiry)	37	%	69	E	101	e
6	ACK (acknowledge)	38	&	70	F	102	f
7	BEL (bell)	39	'	71	G	103	g
8	BS (backspace)	40	(	72	H	104	h
9	HT (horizontal tab)	41	)	73	I	105	i
10	LF (line feed/new line)	42	*	74	J	106	j
11	VT (vertical tab)	43	+	75	K	107	k
12	FF (form feed / new page)	44	,	76	L	108	l
13	CR (carriage return)	45	-	77	M	109	m
14	SO (shift out)	46	.	78	N	110	n
15	SI (shift in)	47	/	79	O	111	o
16	DLE (data link escape)	48	0	80	P	112	p
17	DC1 (data control 1)	49	1	81	Q	113	q
18	DC2 (data control 2)	50	2	82	R	114	r
19	DC3 (data control 3)	51	3	83	S	115	s
20	DC4 (data control 4)	52	4	84	T	116	t
21	NAK (negative acknowledge)	53	5	85	U	117	u
22	SYN (synchronous idle)	54	6	86	V	118	v
23	ETB (end of transmission block)	55	7	87	W	119	w
24	CAN (cancel)	56	8	88	X	120	x
25	EM (end of medium)	57	9	89	Y	121	y
26	SUB (substitute)	58	:	90	Z	122	z
27	ESC (escape)	59	;	91	[	123	{
28	FS (file separator)	60	<	92	\	124	
29	GS (group separator)	61	=	93	]	125	}
30	RS (record separator)	62	>	94	^	126	~
31	US (unit separator)	63	?	95	_	127	DEL (delete)

ASCII Chart

```
main.cpp +
1  #include <iostream>
2
3  int main()
4  {
5      for (char c='A'; c <= '^'; ++c)
6          std::cout << c << ' ';
7  }
```

Ln: 1, Col: 20

[Run](#) [Share](#)

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z [\] ^

Code	Symbol	Code	Symbol	Code	Symbol	Code	Symbol
0	NUL (null)	32	(space)	64	@	96	`
1	SOH (start of header)	33	!	65	A	97	a
2	STX (start of text)	34	"	66	B	98	b
3	ETX (end of text)	35	#	67	C	99	c
4	EOT (end of transmission)	36	\$	68	D	100	d
5	ENQ (enquiry)	37	%	69	E	101	e
6	ACK (acknowledge)	38	&	70	F	102	f
7	BEL (bell)	39	'	71	G	103	g
8	BS (backspace)	40	(	72	H	104	h
9	HT (horizontal tab)	41	)	73	I	105	i
10	LF (line feed/new line)	42	*	74	J	106	j
11	VT (vertical tab)	43	+	75	K	107	k
12	FF (form feed / new page)	44	,	76	L	108	l
13	CR (carriage return)	45	-	77	M	109	m
14	SO (shift out)	46	.	78	N	110	n
15	SI (shift in)	47	/	79	O	111	o
16	DLE (data link escape)	48	0	80	P	112	p
17	DC1 (data control 1)	49	1	81	Q	113	q
18	DC2 (data control 2)	50	2	82	R	114	r
19	DC3 (data control 3)	51	3	83	S	115	s
20	DC4 (data control 4)	52	4	84	T	116	t
21	NAK (negative acknowledge)	53	5	85	U	117	u
22	SYN (synchronous idle)	54	6	86	V	118	v
23	ETB (end of transmission block)	55	7	87	W	119	w
24	CAN (cancel)	56	8	88	X	120	x
25	EM (end of medium)	57	9	89	Y	121	y
26	SUB (substitute)	58	:	90	Z	122	z
27	ESC (escape)	59	;	91	[	123	{
28	FS (file separator)	60	<	92	\	124	
29	GS (group separator)	61	=	93	]	125	}
30	RS (record separator)	62	>	94	^	126	~
31	US (unit separator)	63	?	95	_	127	DEL (delete)

Floating-point types

A floating-point number = $s \times r^j$ (Expressions with exponents)

Significand s with p significant bits

Exponent j

Radix r , and typically $r = 2$

For a fixed radix r , representation of a particular number is not unique if no constraints placed on s and j ; to maximize the number of significant digits in significand, s and j are usually chosen such that first nonzero digit in significand is to **immediate left of radix point**: $1 \leq |s| < r$ (typically $1 \leq |s| < 2$)

Float: 4 Bytes (32 bits)

How to store a float number:



Example:

$$(-1)^{\text{sign}} (b_0. \text{fraction}) \times 2^j, \text{ where } \text{sign} \in \{0,1\}, b_i \in \{0,1\}$$

$$0.75 = 0.5 + 0.25 = 0.11_b = (-1)^0 1.1_b \times 2^{-1}$$

$$1.25 = 1 + 0.25 = 1.01_b = (-1)^0 1.01_b \times 2^0$$

$$-0.5 = -0.1_b = (-1)^1 1.0_b \times 2^{-1}$$

Data types: Allowed operations

- Integer type: **+** **-** ***** **/** **%**

addition, subtraction, multiplication, division, modulo

```
main.cpp +
1  #include<iostream>
2
3  int main()
4  {
5      int a = 5;
6      int b = 3;
7      int c = a % b;
8      std::cout << c << std::endl;
9
10     return 0;
11 }
```

Ln: 1, Col: 19

[Run](#) [Share](#) Command Line Arguments

2

- Floating-point type: **+** **-** ***** **/**
- Character type: like integers (ASCII)

Variable definition

- Definition introduces identifier for variable

Data types → `int` `my_number`; **Identifiers** →
`my_number = 2024;` `int my_number = 2024;`

- Each identifier must be declared before it can be used
- Variables of the same type can be declared together

```
int a, b, c;
```

Assign values to variables

- Initialize a variable

```
int my_number;  
my_number = 2024;
```

Literals

Use "=" for Assignment

```
int my_number = 2024;
```

Literals

- What is value stored inside a variable if it is declared but not initialized?

```
</> ONLINE CPP  
main.cpp +  
1 #include<iostream>  
2  
3 int main()  
4 {  
5     int my_number;  
6     std::cout << my_number;  
7  
8     return 0;  
9 }
```

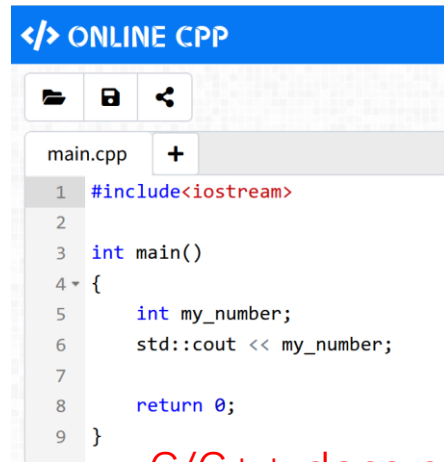
Assign values to variables

- Initialize a variable

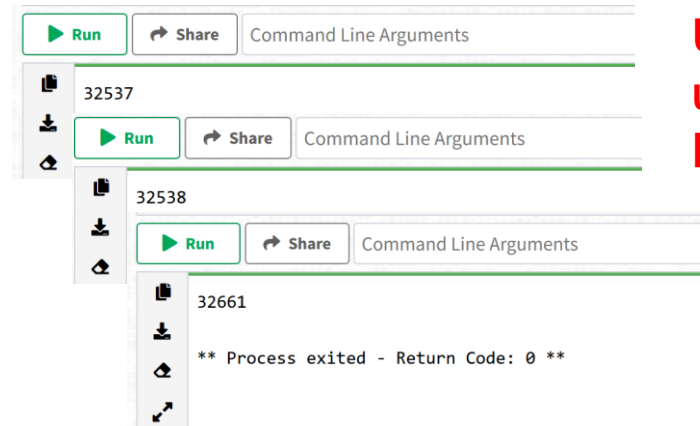
```
int my_number;  
  
my_number = 2024;
```

```
int my_number = 2024;
```

- What is value stored inside a variable if it is declared but **not initialized**?



```
</> ONLINE CPP  
main.cpp  
1 #include<iostream>  
2  
3 int main()  
4 {  
5     int my_number;  
6     std::cout << my_number;  
7  
8     return 0;  
9 }
```



```
Run Share Command Line Arguments  
32537  
Run Share Command Line Arguments  
32538  
Run Share Command Line Arguments  
32661  
** Process exited - Return Code: 0 **
```

Using the values of uninitialized variables can lead to unexpected results

C/C++ does not automatically initialize most variables to a given value, the default value is whatever (garbage) value happens to already be in that memory address!

Assign values to variables

- Simple assignment

```
int a;  
a = 100;
```

← Never forget the semicolon!

- Multiple assignment

```
int a, b, c;  
a = b = c = 100;  
a = (b = (c = 100));
```

- Compound assignment (more details in Chapter 3)

```
int a = 100;  
a += 5;
```

Assign values to variables

- Consistency of data types

```
main.cpp +
1 #include<iostream>
2
3 int main()
4 {
5     double a;
6     a = 1.414f;
7     std::cout << a;
8
9     return 0;
10 }
```

Ln: 1, Col: 1 (19 selected)

Run **Share** Command Line Arguments

1.414

** Process exited - Return Code: 0 **

Inconsistent data type!

```
main.cpp +
1 #include<iostream>
2
3 int main()
4 {
5     int a;
6     a = 1.414f;
7     std::cout << a;
8
9     return 0;
10 }
```

Ln: 1, Col: 19

Run **Share** Command Line Arguments

1

** Process exited - Return Code: 0 *

```
main.cpp +
1 #include<iostream>
2
3 int main()
4 {
5     int a;
6     a = "Hello world";
7     std::cout << a;
8
9     return 0;
10 }
```

Ln: 1, Col: 19

Run **Share** Command Line Arguments

main.cpp: In function 'int main()':
main.cpp:6:9: error: invalid conversion from 'const char*' to 'int' [-fpermissive]
6 | a = "Hello world";
 | ^~~~~~
 | |
 | const char*

** Process exited - Return Code: 1 **

Basic statement

- Simple output to the screen

`#include <iostream.h>:`

make `iostream.h` available to the program.
It contains some C++ statements to make it easy to perform input and output

`using namespace std:`

available `std` available the program, otherwise `std::cout`, `std::endl`

`cout`: sends the data to `cout`

the stream insertion operator `<<` is used to insert a string of characters into the output stream object `cout`

- a string of characters: letters or numbers enclosed in `""`
- The variable name is not in quotation marks

`endl`: go the start of the next line on the screen

```
1  #include <iostream>
2  using namespace std;
3  main()
4  {
5      int v1;
6      float v2;
7      char v3;
8      v1 = 65;
9      v2 = -18.23;
10     v3 = 'a';
11     cout << "Program Example P2B" << endl;
12     cout << "v1 has the value " << v1 << endl;
13     cout << "v2 has the value " << v2 << endl;
14     cout << "v3 has the value " << v3 << endl;
15     cout << "End of program" << endl;
16 }
```

Never forget the semicolon!

```
Program Example P2B
v1 has the value 65
v2 has the value -18.23
v3 has the value a
End of program
```

Basic statement

- Comments
- Comments are added to a C++ program to make it more readable for the programmer
 - completely ignored by the compiler.
 - comments start with the characters `//` and end at the end of the line.
 - C-style comments begin with the characters `/*` and end with the characters `*/` is also used in C++
- Get into the habit of using comments !
- Bad comments can lead to confusion !

```
int v1;    // v1 is an integer variable.  
float v2;  // v2 is a floating-point variable.  
char v3;   // v3 is a character variable.
```

```
/* Program name      : P2A.  
                    Introduction to variables.  
   Written by       : Paul Kelly and SU Xiaohong  
   Date            : 29/6/2009.  
   Version number: 1.0  
*/
```

Basic statement

- Variable definition operations
- Definition introduces identifier for variable

Data types `int my_number;` Identifiers

`int my_number = 2024;`

- Each identifier must be declared before it can be used
- Variables of the same type can be declared together

`int a, b, c;`

Basic statement

- Variable assignment operations
- Initialize a variable

Use “=” for Assignment

```
int my_number;  
my_number = 2024;
```

```
int my_number = 2024;
```

- What is value stored inside a variable if it is declared but not initialized?

```
#include <iostream>  
  
int main()  
{  
    int my_number;  
    std::cout << my_number;  
  
    return 0;  
}
```

- Using the values of uninitialized variables can lead to unexpected results
- C/C++ does not automatically initialize most variables to a given value, the default value is whatever (garbage) value happens to already be in that memory address!

```
32765  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Basic statement

- Variable assignment operations

- Multiple assignment

```
int a, b, c;  
a = b = c = 100;  
a = (b = (c = 100));
```

- Compound assignment(more details in next lecture)

```
int a = 100;  
a += 5;
```

- Check the consistency of data types

```
#include <iostream>  
  
int main()  
{  
    double a;  
    a = 1.414f;  
    std::cout << a;  
    return 0;  
}
```

```
#include <iostream>  
  
int main()  
{  
    int a;  
    a = 1.414f;  
    std::cout << a;  
    return 0;  
}
```

```
#include <iostream>  
  
int main()  
{  
    int a;  
    a = "hello world";  
    std::cout << a;  
    return 0;  
}
```

HOMEWORK

Homework 2

- 1. Which of the following are valid C++ variable names?
If valid, do you think the name is good mnemonic (i.e. reminds you of its purpose)?
 - (a) stock_code (b) money\$ (c) Jan_Sales (d) X-RAY
 - (e) int (f) xyz (g) la (h) invoice_total
 - (i) John's_exam_mark
- 2. Which of the following are valid variable definitions?
 - (a) integer account_code; (b) float balance; (c) decimal total;
 - (d) int age; (e) double int;

Homework 2

- 3. Write variable definitions for each of the following:
 - (a) integer variables `number_of_transactions` and `age_in_years`
 - (b) floating-point variables `total_pay`, `tax_payment`, `distance` and `average`
 - (c) long integer variables `record_position` and `count`
 - (d) a character variable `account_type`
 - (e) a double variable `gross_pay`

Homework 2

- 4. Write a C++ program to assign values to the variables in exercise 3 and display the value of each variable on a separate line.

Basics syntax of C++

Summary

- Variables
- Constants
- Memory and size
- Basic statements

The syntax of variables

- To store data in C++: use Variables, Constants
- Variables: can change its values in a program
- Example:
 1. Main() //every C++ program must have a main function for execution
 2. { //the program statements are within the brace { and }.
 3. int v1; //define the name and type of variables
 4. float v2; //define the name and type of variables
 5. char v3; //define the name and type of variables
 6. v1=65; //assign integer values to the variable v1
 7. v2=-18.23; //assign float values to the variable v2
 8. v3='a'; //assign character to the variable v3
 9. v1=14; //value of variables can be changed
 10. }

The syntax of variables

- A variable must be defined before it can be used. A variable is defined by giving it a data type and a name

```
int age = 25;
```

- A variable can be given any name, called an identifier in C++, provided it is within the following rules
 - An identifier can only be constructed using letters, numerals or underscores (_).
 - An identifier must start with a letter or an underscore.
 - An identifier cannot be a C++ keyword. A keyword is a word that has a special meaning, e.g. `int int/float=25` is forbidden.
 - An identifier can contain any number of characters, but only the first thirty-one characters are significant to the C++ compiler.
- A variable is like a box in the memory of the computer.

ADDITIONAL READING

Data type conversion

- Automatic conversion

```
int a = 100;  
double b = 100.05;
```

b = a; Conversion from **int** to **double**

a = b; Conversion from **double** to **int** **Data Loss!**

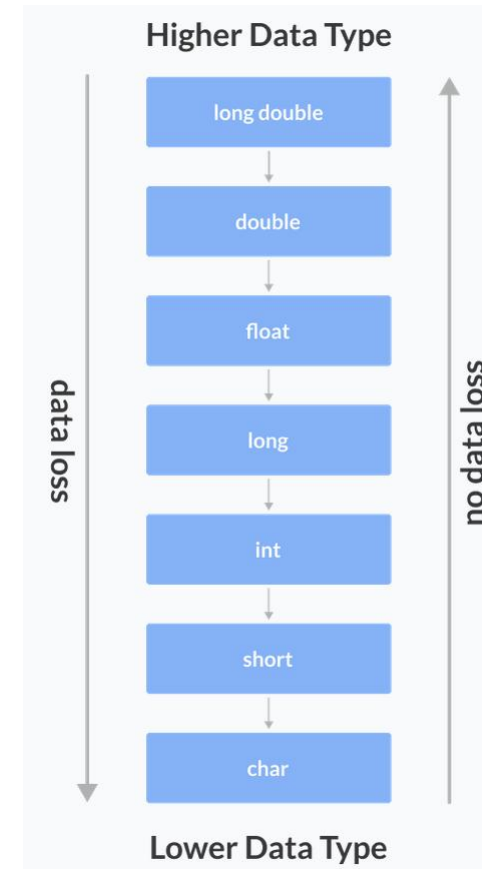
Data type conversion

- Automatic conversion

```
int a = 100;  
double b = 100.05;
```

b = a; Conversion from **int** to **double**

a = b; Conversion from **double** to **int** **Data Loss!**



Data type conversion

- Explicit conversion

```
int a = 100;  
double b = 100.05;
```

```
a = (int)b;      C-style Type Casting
```